AI Compiler Performance Engineering

Auto-Loops Books — ch20

Contents

1 Mirage and Emerging AI Compilers **1**

1.1 Mirage HW matrix 1

1.2 Mirage arch passes 5

1.3 Mirage codegen 8

1.4 Mirage tuning 10

1.5 Mirage case study 13

1.6 Key Takeaways 16

1.7 Conclusion 18

Bibliography **19**

Index **22**

Chapter 1

Mirage and Emerging AI Compilers

The compilers of Chapters 16–19 all answer the same question—given this operator, schedule it well—but Mirage asks a different one: given this whole tensor program, *superoptimize* it, discovering optimizations that span algebraic rewrites, schedules, and entirely new custom kernels at once (Wu *et al.*, 2024). That shift matters for decode because the highest-leverage transformation is not a better matmul tile; it is collapsing a token's worth of operators into one kernel so the host stops launching thousands of them. Dense Llama-3.2-1B averages 847.5 GPU launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Mirage's descendant, the Mirage Persistent Kernel (MPK), takes this to its conclusion by transforming LLM inference into a single megakernel and cutting end-to-end latency by 1.2–6.7× (Mirage Project contributors, 2025). This chapter is also where the book's own reference compiler comes from: **YiRage** is a Mirage-lineage, multi-backend descendant, so the ideas here—multi-level search, megakernel fusion, residency-as-metadata—are the spine of everything we have called YiRage in earlier chapters (Chen and YiRage Contributors, 2026). We treat Mirage and the emerging wave as production surfaces graded on bytes/token and launches/token, not as research curiosities (Chen, 2026; Davies *et al.*, 2025).

1.1 Mirage HW matrix

Mirage's "**matrix**" is not a list of vendor targets like IREE's; it is the GPU compute hierarchy itself (Wu *et al.*, 2024). A μGraph represents a tensor program

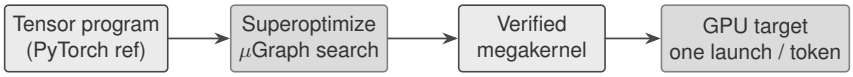


Figure 1.1: Mirage superoptimizes a tensor program over a multi-level μGraph search space and emits a verified, fused kernel—one launch per token in the MPK limit.

uniformly at three levels—kernel, thread block, and thread—and that hierarchy *is* the **hardware** model the superoptimizer searches against (Wu *et al.*, 2024). The consequence is that Mirage is GPU-first by construction: the kernel/block/thread structure maps directly onto a CUDA **target**, and its strongest results are on NVIDIA GPUs (Mirage Project contributors, 2025).

The honest multi-hardware reading follows from that. On a **gpu** Mirage is exactly aligned with the residency hierarchy—block-level tensors live in shared memory, kernel-level tensors in device memory—so its search naturally optimizes bytes/token (Wu *et al.*, 2024; Chen, 2026). On a **cpu** the kernel/block/thread abstraction does not map cleanly, so the superoptimization approach is emerging rather than production there (Goodfellow *et al.*, 2016). On an **npu** like XDNA/AIE the static-dataflow model is different again, and the value Mirage offers is conceptual—multi-level search and verified fusion—rather than a ready backend (AMD, 2024b,a).

Mirage is also a representative of a broader emerging wave, and the family resemblance is what matters more than any one project. The common thread across the new compilers is a move away from fixed, hand-authored pass pipelines toward *search with verification*: instead of trusting a human-proved rewrite, these systems explore a large space of equivalent programs and certify the winner, which lets them discover fusions and custom kernels that no pass library encodes (Wu *et al.*, 2024). The second shared theme is end-to-end fusion as a first-class goal rather than a peephole optimization—the megakernel is the clearest example, but the same instinct drives whole-program scheduling in IREE and aggressive operator fusion elsewhere (Mirage Project contributors, 2025; IREE / iree-org contributors, 2024). For a decode engineer the practical signal is that the frontier has shifted from "which library has the fastest GEMM" to "which compiler can collapse my whole token into the fewest, most resident launches and prove it correct" (Vellaisamy *et al.*, 2026; Chen, 2026).

Multi-hardware delta. A Hopper GPU rewards the megakernel fusion Mirage discovers because it removes launches and keeps tensors in shared memory; a static NPU fabric needs the fusion decided at compile time with a fixed tile/DMA

plan instead (SemiAnalysis, 2024; AMD, 2024a). This is precisely why YiRage generalizes the Mirage idea across backends rather than shipping a single GPU artifact (Chen and YiRage Contributors, 2026).

1.1.1 Placement in the compiler family

Mirage belongs to the search-based branch of the compiler family, alongside TVM's AutoTVM and Ansor, but it differs in the object of search (Wu *et al.*, 2024; Chen *et al.*, 2018a; Zheng *et al.*, 2020). Where TVM searches schedules for operators whose math is fixed, Mirage searches the math itself, the schedule, and the kernel structure in one space (Wu *et al.*, 2024; Chen *et al.*, 2018b). That is why its discoveries can look like entirely new kernels rather than faster versions of known ones (Wu *et al.*, 2024; Davies *et al.*, 2025). For a decode engineer the practical consequence is that Mirage is not a replacement for the graph compilers of the previous chapters; it is a deeper optimizer that can be pointed at the fused regions those compilers produce (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The evaluation should therefore ask not whether Mirage compiles the whole model, but whether the regions it superoptimizes are the regions where the token spends its bytes and launches (Chen, 2026; Davies *et al.*, 2025).

1.1.2 Where emerging compilers pay off

Superoptimization and verified megakernels pay off only on programs that are compiled once and executed many times, because the search cost is amortized over executions (Wu *et al.*, 2024; Kwon *et al.*, 2023). A served LLM's decoder layers are the canonical example: the same layer graph runs on every token of every request, so a long search that finds a better fused kernel repays itself within hours of production traffic (Vellaisamy *et al.*, 2026; Chen, 2026). One-off graphs, preprocessing pipelines, and rapidly changing research code do not justify the same budget (Wu *et al.*, 2024; Chen, 2020). The deployment rule is to compile hot regions with the deepest optimizer available and cold regions with the fastest conventional path (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). This hybrid is how a production stack can afford emerging-compiler techniques without making every build slow (Wu *et al.*, 2024; Chen, 2026).

1.1.3 Reproducibility of search results

Search-based compilation introduces a reproducibility obligation: the same source can produce different kernels depending on search time, random seed, hardware

state, and library version (Wu *et al.*, 2024; Chen, 2020). A serving artifact must therefore record the search configuration, the verification report, and the measured reward of the accepted candidate (Chen, 2026; Davies *et al.*, 2025). Reproducing a regression means re-running the same search with the same budget and seed, not merely recompiling (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Teams should archive the accepted μGraph and the emitted kernel beside the benchmark so the compiled artifact is never a black box (Chen, 2020, 2026). Search reproducibility is the price of admitting search into the production toolchain, and it must be paid in the release process rather than discovered during an incident (Wu *et al.*, 2024; Davies *et al.*, 2025).

1.1.4 Evaluation plan for an emerging compiler

Before adopting any emerging compiler in a decode stack, run a bounded evaluation with the same shape as this chapter's gate: pick the served model and SKU, compile the hot decoder layers with the new tool and with the incumbent path, and compare launches/token, bytes/token, and ms/token at every shipped context bucket (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026). The evaluation must also record compile time, verification time, and artifact size, because those decide whether the approach is operationally viable (Wu *et al.*, 2024; Davies *et al.*, 2025). Include a rollback test that restores the incumbent path and confirms identical behavior (Chen, 2020, 2026). An emerging compiler that wins the kernel benchmark but cannot produce a verifiable, rollback-capable artifact is not ready for serving regardless of its speedups (Davies *et al.*, 2025; Wu *et al.*, 2024).

1.1.5 Adoption maturity criteria

An emerging compiler deserves production adoption only when it meets the same maturity bar as the established surfaces: reproducible builds, verification artifacts, bounded compile time, rollback paths, and per-SKU regeneration (Wu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026). Research demos and published speedups are evidence of potential, not evidence of operational readiness (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The adoption checklist should mirror this chapter's evaluation plan and add the maintenance dimension: who regenerates kernels when the model changes, and how long does that take (Chen, 2020; Wu *et al.*, 2024). A compiler that cannot be operated by the serving team is not an improvement regardless of its peak results (Davies *et al.*, 2025; Chen, 2026). Maturity criteria keep the book's yardstick honest as the field advances (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

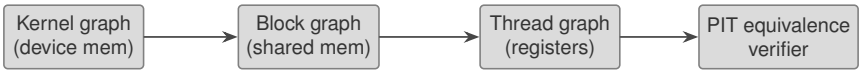


Figure 1.2: Mirage's μGraph spans kernel (device memory), block (shared memory), and thread (register) levels; candidates are checked by a probabilistic equivalence verifier.

Table 1.1: Mirage's μGraph levels and the memory tier each one keeps tensors in; the level structure is the residency contract (Wu *et al.*, 2024).

Level	Memory tier	Role
Kernel graph	Device memory (HBM)	Whole-GPU kernels; tensors shared via DRAM
Block graph	Shared memory	Thread-block compute; intermediates on-chip
Thread graph	Registers	Per-thread scope; finest reuse

1.2 Mirage arch passes

Mirage's "**pass**" pipeline is really a search, and that is the conceptual departure from every prior chapter (Wu *et al.*, 2024). Instead of a fixed sequence of rewrites, the superoptimizer explores a space of μGraphs that jointly combine algebraic transformations, schedule transformations, and the generation of new custom kernels—an **optim**ization no single hand-written pass would find (Wu *et al.*, 2024). The μGraph levels carry the residency contract directly: a kernel graph keeps tensors in device memory, a block graph keeps intermediates in shared memory to cut device-memory access, and a thread graph **lower**s scope to registers (Wu *et al.*, 2024).

The level structure is not bookkeeping; it is the whole reason the search can reason about bytes/token. When the optimizer chooses to express a sub-computation as a block graph, it is committing those intermediates to shared memory and therefore removing the device-memory traffic a kernel-level expression would incur (Wu *et al.*, 2024; Chen, 2026). When an intermediate is too large for shared memory, Mirage splits the work into multiple block graphs so that each one fits, which keeps the on-chip residency invariant intact instead of silently spilling (Wu *et al.*, 2024). A human writing CUDA makes these decisions implicitly and inconsistently; the μGraph makes them explicit and searchable, which is what lets the superoptimizer trade device-memory traffic for shared-memory reuse deliberately.

Two ideas make this tractable and trustworthy. First, abstraction-based pruning shrinks the enormous search space while preserving a certain optimality guarantee, so the search is not brute force (Wu *et al.*, 2024). Second, a probabilistic equivalence

verifier checks each candidate μGraph against the reference by random testing over finite fields—polynomial identity testing generalized to the LAX fragment (linear algebra plus division and exponentiation)—which avoids floating-point noise and drives the probability of accepting a wrong kernel arbitrarily low (Wu *et al.*, 2024). This is a different correctness model from a compiler that trusts each hand-proved rewrite, and it is what lets Mirage emit kernels a human never wrote. Notably, Mirage does not lower through an MLIR **dialect** tower; the multi-level μGraph is a single uniform representation, which is why it can search across levels that a staged dialect pipeline keeps separate (Lai *et al.*, 2023; Wu *et al.*, 2024).

What the verifier buys. Because equivalence is checked rather than assumed, a discovered megakernel can be deployed with quantified confidence, not just empirical testing—the discipline a decode team needs before replacing a known-correct kernel-per-operator path with a fused one (Wu *et al.*, 2024; Davies *et al.*, 2025).

1.2.1 Search space and pruning guarantees

The μGraph search space is too large for brute force, and Mirage's abstraction-based pruning is what makes the search practical while preserving a formal optimality property (Wu *et al.*, 2024). The pruning works over the algebraic structure of the graph rather than by random sampling, so the search can discard whole equivalence classes without enumerating them (Wu *et al.*, 2024; Chen, 2020). For decode, the useful consequence is that the optimizer can reason about layers at the granularity of attention and MLP blocks instead of individual matmuls (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). The search should therefore be seeded with the layer structure the model actually has, so the optimizer explores megakernel arrangements rather than re-deriving the layer decomposition from scratch (Wu *et al.*, 2024; Davies *et al.*, 2025). Teams should record which pruning assumptions held in the compile report, because a future model whose structure violates them may silently narrow the effective search (Chen, 2026; Wu *et al.*, 2024).

1.2.2 Verification coverage boundaries

The probabilistic verifier certifies candidates inside the LAX fragment—linear algebra plus division and exponentiation—and the guarantee is scoped to that fragment (Wu *et al.*, 2024). Operators outside the fragment need a different correctness argument, which means the deployable confidence of a fused kernel is only as broad as the fragment that generated it (Wu *et al.*, 2024; Davies *et al.*,

2025). A decode layer mixes in-fragment operations (matmul, softmax-related exponentials, elementwise arithmetic) with framework behavior that may not be covered, such as sampling or custom kernels (Dao *et al.*, 2022; Mirage Project contributors, 2025). The deployment checklist should therefore enumerate which parts of the fused kernel are verifier-certified and which rest on conventional testing (Chen, 2020, 2026). This boundary is not a weakness; it is a precise statement of where the quantified confidence ends and where empirical validation must begin (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

1.2.3 Verification in the release pipeline

The equivalence check should run as a first-class step in CI for every accepted candidate, alongside the byte and launch benchmarks (Wu *et al.*, 2024; Davies *et al.*, 2025). The verification report should state the fragment coverage, the number of random tests, and the resulting bound on accepting a wrong kernel (Wu *et al.*, 2024; Chen, 2020). A model or compiler upgrade that changes the candidate should trigger a fresh verification before the artifact is released (Chen, 2026; Vellaisamy *et al.*, 2026). Because the verifier is probabilistic, the CI configuration should choose the test budget to meet the serving team's correctness bar, and that bar should be documented in the runbook (Wu *et al.*, 2024; Davies *et al.*, 2025). Teams that treat verification as an optional research step will eventually ship a search artifact whose correctness they merely hope for (Chen, 2026; Wu *et al.*, 2024).

1.2.4 Choosing megakernel boundaries

The megakernel boundary is a search decision with operational consequences: fusing too little leaves launch overhead, while fusing too much risks spills and long device-side dependencies (Wu *et al.*, 2024; Mirage Project contributors, 2025; Chen, 2026). The search should be allowed to propose boundaries that match layer structure—attention core, attention plus projections, or the full decode layer—and the accepted boundary should be the one that minimizes measured ms/token at the served bucket without violating on-chip budgets (Dao *et al.*, 2022; Wu *et al.*, 2024). A boundary that looks elegant in the IR but produces a long serial dependency chain may be slower than a two-kernel split that overlaps better (Mirage Project contributors, 2025; Davies *et al.*, 2025). The compile report should justify the chosen boundary with the task graph, not with a naming convention (Wu *et al.*, 2024; Chen, 2026). Decode teams should treat boundary selection as a benchmarked parameter that changes per model and SKU (Vellaisamy *et al.*, 2026; NVIDIA, 2022).

1.3 Mirage codegen

Mirage's **codegen** turns a verified μGraph into CUDA, and MPK extends this to **emit** an entire LLM forward pass as one megakernel (Wu *et al.*, 2024; Mirage Project contributors, 2025). The mechanism is an SM-level task graph (tGraph) that captures data dependencies at the granularity of individual streaming multiprocessors, so the **backend** can schedule cross-operator software pipelining and communication–computation overlap *inside* a single kernel launch, with decentralized scheduling across SMs (Mirage Project contributors, 2025). The **platform** is the GPU, and the generated CUDA includes the intra-SM optimizations—software pipelining, register reuse, bank-conflict-aware layouts—that a per-operator emit cannot coordinate across operators (Mirage Project contributors, 2025).

For decode this is the most direct attack on launches/token in the book. A kernel-per-operator stack pays host dispatch for every one of the thousands of launches a token requires; a megakernel pays it once (Vellaisamy *et al.*, 2026; Chen, 2026). MPK reports 1.2–6.7× end-to-end latency reductions from exactly this collapse, pushing inference toward hardware limits, and it does so from a few dozen lines of Python that name the fused layers—for example one call fusing RMSNorm and a linear projection (Mirage Project contributors, 2025). What codegen still cannot fix is an algorithm that is genuinely device-memory bound: if the working set does not fit on chip, even a perfect megakernel must stream it (Goodfellow *et al.*, 2016).

The deeper reason the megakernel helps is that it changes what can overlap. In a kernel-per-operator schedule, the boundary between two operators is also a synchronization point: the host waits for one kernel to finish before launching the next, so the GPU idles through dispatch latency and any tail effects (Vellaisamy *et al.*, 2026). Inside a single megakernel those boundaries become intra-kernel dependencies that the SM-level task graph can pipeline—one SM can start the next task's loads while another finishes the current task's compute, and collective communication can overlap with computation instead of stalling behind it (Mirage Project contributors, 2025). At batch-1 decode, where there is no batch dimension to hide latency behind, this fine-grained overlap is often the only source of parallelism left, which is why the megakernel wins are largest exactly in the regime that hurts a conventional stack most (Chen, 2026; Kwon *et al.*, 2023).

Multi-hardware delta. The megakernel's single-launch win is a GPU phenomenon—it removes CUDA launch and host-dispatch overhead; on an NPU the equivalent is a static single-dispatch schedule, and on a CPU it is the AOT bundle of Chapter 19

(AMD, 2024a; Rotem *et al.*, 2018). Same principle, different mechanism per class (SemiAnalysis, 2024).

1.3.1 SM-level task graph scheduling

The tGraph moves scheduling from the host into the device by expressing dependencies at SM granularity, which is what makes intra-kernel pipelining and overlap possible (Mirage Project contributors, 2025). A task graph node corresponds to work an SM can start once its inputs are ready, and the scheduler keeps SMs busy by advancing independent tasks while a dependency resolves (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026). For decode, this replaces the kernel-boundary stalls of a per-operator stack with fine-grained producer-consumer overlap (Mirage Project contributors, 2025; Chen, 2026). The codegen review should inspect the task graph for serialization: a long chain of dependent tasks with no overlap opportunity indicates that fusion created a schedule but not a pipelined one (Davies *et al.*, 2025; Mirage Project contributors, 2025). The tGraph is also where communication tasks attach, so collective and copy operations appear as first-class graph nodes whose latency the scheduler can hide (Mirage Project contributors, 2025; SemiAnalysis, 2024).

1.3.2 Register and shared-memory budgets during codegen

Emitting a megakernel concentrates every intermediate into the same kernel, so the codegen stage must respect the GPU's register file and shared-memory capacity while scheduling the task graph (Mirage Project contributors, 2025; NVIDIA, 2022). When the fused working set exceeds those budgets, the emitter either spills or splits the megakernel into sub-kernels, and the split point becomes a new launch boundary (Chen, 2026; Vellaisamy *et al.*, 2026). The compile report should state the peak shared-memory and register usage of the accepted kernel and compare them to the device limits (Wu *et al.*, 2024; Davies *et al.*, 2025). A megakernel that only fits by spilling may lose the launch-collapse benefit, because the spill traffic can exceed the dispatch overhead it removed (Chen, 2026; Mirage Project contributors, 2025). The right fusion level is therefore not the maximum the search can express but the maximum that fits the target without spilling, measured at the decode shape (NVIDIA, 2022; Chen, 2026).

1.3.3 Portability across GPU generations

Mirage's accepted kernels are generated against a concrete hardware model—SM count, shared-memory capacity, register budget, and async-copy behavior (Wu *et al.*, 2024; NVIDIA, 2022). Moving the same model to a different GPU generation requires regenerating and re-verifying the kernel rather than reusing the artifact (Wu *et al.*, 2024; Davies *et al.*, 2025). A fleet with mixed GPUs therefore needs per-SKU search artifacts and a dispatch layer that selects the matching kernel (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Driver changes can also invalidate an artifact if they alter scheduling behavior or instruction selection, so the artifact manifest must pin the driver and CUDA version used during search and codegen (Chen, 2020, 2026). Portability in the Mirage world means the reference program is portable and the search re-runs per target; it does not mean the emitted kernel is universal (Wu *et al.*, 2024; NVIDIA, 2022).

1.4 Mirage tuning

In Mirage, **tun**ing and compilation are the same act: the superoptimizer searches the μGraph space and the probabilistic verifier accepts only equivalent candidates, so "tuning" means letting the search run rather than hand-listing configurations (Wu *et al.*, 2024). You still **profile** candidates—the search is guided by measured performance and bounded by the abstraction-based pruning—but the engineer's job is to set the search budget and the operating point, not to write schedules (Wu *et al.*, 2024).

The **pitfall** is twofold and specific. First, superoptimization is expensive: searching and verifying a large μGraph space costs real compile time, so it pays for hot, reused programs (a served LLM's decode layer) and not for one-off graphs (Wu *et al.*, 2024; Kwon *et al.*, 2023). Second, the correctness guarantee is probabilistic and scoped to the LAX fragment; operators outside that fragment do not get the same verification strength, so a team must know which parts of its model the guarantee actually covers (Wu *et al.*, 2024). As elsewhere, the right search target is **hardware**-specific—a μGraph tuned for one GPU's shared-memory size and SM count is not automatically optimal on another, so re-search per SKU rather than assuming transfer (NVIDIA, 2022; Davies *et al.*, 2025). A practical way to budget the search is to spend it where the token spends its time: the decode-critical attention and projection layers are reused on every step and every request, so a long superoptimization run there amortizes over billions of tokens, whereas one-off preprocessing graphs rarely justify the cost (Wu *et al.*, 2024; Kwon *et al.*, 2023).

1.4.1 Budgeting search across a model

The search budget should follow where the token spends its time, not where the graph is largest (Wu *et al.*, 2024; Kwon *et al.*, 2023). Attention and projection layers execute once per layer per token, so their superoptimization amortizes immediately; sampling, tokenization, and rare routing paths do not justify long searches (Vellaisamy *et al.*, 2026; Chen, 2026). The budget ledger should record search seconds per region, the reward improvement found, and the measured wall-clock cost of the search (Wu *et al.*, 2024; Davies *et al.*, 2025). When a model changes, only the affected regions need re-search if the accepted artifacts for unchanged regions are cached and still valid for the target (Chen, 2020; Wu *et al.*, 2024). Teams should also bound search by an acceptance threshold: stop when additional search time stops improving the measured decode counters, because the marginal benefit of the last few hours is often negligible (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Budgeting turns an open-ended superoptimizer into a production tool with predictable compile cost (Chen, 2026; Davies *et al.*, 2025).

1.4.2 Measured reward and candidate selection

Search candidates must be ranked by the same counters the rest of the book uses: launches per token, bytes per token, and ms/token at the served shape (Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A candidate that wins an isolated kernel benchmark but raises bytes/token when composed into the layer should be rejected even if its local numbers look better (Wu *et al.*, 2024; Chen, 2026). The reward measurement should run the candidate exactly as serving will, inside the same launch context and with the same input pointers (Davies *et al.*, 2025; Mirage Project contributors, 2025). When two candidates are close in reward, prefer the one with the smaller shared-memory footprint and simpler task graph, because those properties predict robustness across context buckets (Wu *et al.*, 2024; Chen, 2026). The accepted candidate's reward row belongs in the release manifest next to the verification report (Chen, 2020; Vellaisamy *et al.*, 2026).

1.4.3 Caching results across model families

Different models share structural subgraphs—attention blocks, normalization-plus-linear chains—and cached search results for a shared structure can save hours of recompilation (Wu *et al.*, 2024; Dao *et al.*, 2022). The cache key must include the subgraph structure, data types, tensor shapes, and target description, because any of those changes can invalidate the accepted kernel (Wu *et al.*, 2024; Davies

et al., 2025). Reusing a cached kernel for a new model still requires verification against that model's reference, since equivalence is a property of the candidate and the reference program together (Wu *et al.*, 2024; Chen, 2020). A cache that skips verification is not a cache; it is a correctness risk (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Well-designed caches make superoptimization economical across a model zoo, which is the deployment condition that makes the approach sustainable (Davies *et al.*, 2025; Chen, 2026).

1.4.4 Nightly revalidation cadence

Discovered artifacts decay on the same triggers as every other compiled output: model updates, GPU driver changes, CUDA upgrades, and new context buckets all require regeneration and re-verification (Wu *et al.*, 2024; Chen, 2026; Davies *et al.*, 2025). A nightly job should re-run the accepted kernels against the current reference at the served buckets and alert on counter drift or numerical divergence (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The alert triggers a bounded re-search seeded with the previous accepted candidate, which converges faster than a cold search (Wu *et al.*, 2024; Chen, 2020). Full re-verification belongs in the release pipeline whenever the artifact manifest changes (Wu *et al.*, 2024; Chen, 2026). This cadence converts superoptimization from a one-time research deliverable into a maintained production contract (Davies *et al.*, 2025; Wu *et al.*, 2024).

1.4.5 Building the hot-region list

The search budget should be driven by an explicit hot-region list derived from a decode profile: rank every subgraph by its contribution to per-token time and bytes, then assign superoptimization only to the top regions that dominate the total (Wu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026). The list should be regenerated when the model or the serving shape changes, because a region that was hot at one context length may not be at another (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Each region on the list gets a search budget proportional to its measured share, which prevents the optimizer from spending hours on a chain that contributes a fraction of a percent (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). The hot-region list and the budget ledger belong in the release manifest, making the search strategy itself a reviewable artifact (Chen, 2020, 2026). This is how a serving team keeps an expensive superoptimizer pointed at the bytes and launches that actually move the token (Wu *et al.*, 2024; Davies *et al.*, 2025). Update the list when the traffic mix changes, because hot regions track the workload and the context distribution as much as they track the model (Davies *et al.*, 2025; Chen, 2026).

1.5 Mirage case study

The decisive **case** is MPK compiling a real LLM—an open Hugging Face model such as Qwen3-8B—into a single megakernel and **benchmark**ing it against a kernel-per-operator serving stack (Mirage Project contributors, 2025). The result restates the book's thesis from the superoptimizer's side: fusing the whole decode step into one launch removes the per-operator host dispatch that dominates batch-1, and the SM-level task graph overlaps the communication and computation that a kernel-per-operator schedule serializes, yielding the reported 1.2–6.7× (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026). The arithmetic is familiar—each avoided inter-operator round trip saves the activation bytes that would otherwise stream through device memory per token—only now the compiler discovers the fusion rather than a human writing it (Chen, 2026; Dao *et al.*, 2022).

It is worth being concrete about why the search, not just the fusion, is the contribution. A hand-written megakernel for one model is a heroic engineering effort that does not transfer to the next architecture; Mirage's superoptimizer regenerates the fused kernel from a reference implementation whenever the model or the GPU changes, and the probabilistic verifier certifies each result (Wu *et al.*, 2024; Mirage Project contributors, 2025). For a serving team that means the megakernel is not a brittle artifact to maintain by hand but a compiler output to regenerate, which is the only way end-to-end fusion is sustainable across a model zoo that changes monthly. The cost, as the tuning section noted, is search time—paid once per model-target pair and amortized over every token served.

The **cross-hardware** conclusion is where the emerging wave and this book converge. Mirage proves the GPU case; the open problem is carrying multi-level search and verified megakernel fusion to CPUs, NPUs, and heterogeneous deployments where the launch-collapse mechanism differs (AMD, 2024b,a). **YiRage** is the book's answer: a Mirage-lineage compiler that generalizes superoptimization and megakernel fusion across multiple backends and carries the Chapter 2 residency rules as checkable IR metadata, so a discovered fusion that would spill on a given target is rejected at compile time rather than shipped (Chen and YiRage Contributors, 2026). That is why every prior chapter measured its compiler against the same YiRage yardstick—this is where the yardstick comes from (Davies *et al.*, 2025).

1.5.1 Worked RMSNorm-plus-linear fusion

The smallest MPK example is representative: a call that fuses RMSNorm and the following linear projection into one kernel (Mirage Project contributors, 2025). A kernel-per-operator version reads the hidden state, computes normalization statistics, writes the normalized tensor, and reads it back for the projection; the fused version carries the normalized values on-chip between the two operations (Mirage Project contributors, 2025; Chen, 2026). At hidden size d=4096 in BF16, that removes one full-width write and one read per layer per token (Chen, 2026; Vellaisamy *et al.*, 2026). The win is small per layer and large across depth, which is why it matters only when measured over the whole model at the decode shape (Dao *et al.*, 2022; Davies *et al.*, 2025). The worked example should be reproduced on the serving model and SKU before trusting the published numbers, because the search and codegen are hardware-specific (Wu *et al.*, 2024; NVIDIA, 2022).

1.5.2 Verification workflow in serving

Promoting a discovered megakernel into serving should follow a fixed workflow: run the probabilistic verifier on the candidate, record the fragment coverage and confidence, benchmark the candidate at the served buckets, and only then replace the reference path behind a runtime switch (Wu *et al.*, 2024; Davies *et al.*, 2025). The switch should keep the reference path available so a field regression can fall back without redeployment (Mirage Project contributors, 2025; Chen, 2026). Telemetry should tag which kernel variant ran for each request so the megakernel's actual behavior is observable in production (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Because the verifier is probabilistic, the serving team should also keep a bounded-error numerical check against the reference on real traffic as a second line of defense (Wu *et al.*, 2024; Chen, 2026). This workflow converts a research-strength verifier into a deployable confidence gate (Wu *et al.*, 2024; Chen, 2020).

1.5.3 Autopsies for megakernel regressions

Megakernel failures have distinctive signatures. A latency regression with correct output usually means the fused kernel spills at the served context bucket even though it fit the search shape; the fix is to re-search with the real bucket's footprint (Mirage Project contributors, 2025; Chen, 2026). A numerical drift usually means part of the fused graph fell outside the verified fragment or the verifier budget was too small; the fix is to expand coverage or testing (Wu *et al.*, 2024; Davies *et al.*, 2025). A regression after a driver or GPU change usually means the artifact

was not regenerated for the new environment, which the manifest should have prevented (Chen, 2020; Vellaisamy *et al.*, 2026). Each autopsy starts from the verification report and the artifact manifest, making the diagnosis a bounded check rather than a rewrite (Wu *et al.*, 2024; Chen, 2026). Teams that keep the reference path and telemetry in place can treat megakernel incidents as configurable events instead of emergencies (Davies *et al.*, 2025; Mirage Project contributors, 2025).

1.5.4 The memory-bound limit

Megakernel fusion removes launches and inter-operator round trips, but it cannot create on-chip capacity that does not exist (Wu *et al.*, 2024; Chen, 2026). When a model's working set—weights, KV, and activations—exceeds the GPU's on-chip storage at the served context bucket, the fused kernel must stream from HBM and the fusion win shrinks toward the memory floor (Chen, 2026; Vellaisamy *et al.*, 2026). The case study should therefore report the ratio of working set to on-chip capacity at each bucket, because that ratio predicts how much of the megakernel's theoretical win is reachable (Wu *et al.*, 2024; Davies *et al.*, 2025). At long context, KV streaming dominates and the fusion of the attention loop remains valuable even when whole-layer fusion is no longer profitable (Dao *et al.*, 2022; Mirage Project contributors, 2025). Knowing the memory-bound limit keeps expectations honest: the megakernel is the best possible schedule of the bytes that must move, not a substitute for moving fewer bytes (Chen, 2026; Wu *et al.*, 2024).

1.5.5 Transitioning from incumbent kernels

Replacing a known-good kernel-per-operator path with a discovered megakernel should be staged like any risky deployment (Mirage Project contributors, 2025; Davies *et al.*, 2025). Run shadow traffic through both paths, compare outputs under a bounded-error metric, and verify that the latency win survives the real request mix before switching production traffic (Wu *et al.*, 2024; Chen, 2026). The transition window should keep telemetry on both paths so any difference is attributable to the artifact change (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Rollback must be a runtime switch rather than a redeploy, because a megakernel incident discovered in the field needs a fast escape hatch (Mirage Project contributors, 2025; Chen, 2020). The staged transition is also the best evidence the search reward function matches the production objective, since a kernel that wins the search benchmark but loses shadow traffic reveals a reward mismatch (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

1.5.6 Cross-backend generalization

The megakernel's mechanism is GPU-specific, but the ideas generalize to other hardware classes with different mechanisms (Wu *et al.*, 2024; AMD, 2024b; Rotem *et al.*, 2018). An NPU needs the fused schedule fixed at compile time as a static dataflow graph; a CPU needs the AOT bundle with stacked kernels; both are forms of end-to-end fusion with verification, expressed in the machine model of each class (AMD, 2024b; Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Emerging compilers are beginning to explore these translations, and the evaluation criterion is always the same: launches or dispatch count falls, bytes moved falls, and the artifact is verified against the reference (Wu *et al.*, 2024; Chen, 2026). YiRage embodies this direction by generalizing search and megakernel ideas across backends while keeping residency rules as checkable metadata (Chen and YiRage Contributors, 2026). Teams should watch emerging compilers for cross-backend support but should not assume the GPU result transfers until it is verified on their target (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

1.5.7 Caveats about probabilistic verification

Probabilistic verification is powerful but has limits that deployment must respect (Wu *et al.*, 2024; Davies *et al.*, 2025). The confidence bound assumes the random testing configuration is correct, so the CI configuration must be part of the artifact record (Wu *et al.*, 2024; Chen, 2020). Verification covers the fragment the verifier understands; operators, dtypes, or control flow outside it need conventional testing with real traffic (Wu *et al.*, 2024; Chen, 2026). Floating-point behavior adds another layer: equivalence over finite fields proves algebraic identity, not bitwise identity, so a bounded numerical tolerance check remains necessary in production (Dao *et al.*, 2022; Wu *et al.*, 2024). Teams should document the combination of probabilistic verification and numerical testing that a release actually received, because the two together form the correctness argument (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026).

1.6 Key Takeaways

- **Superoptimization replaces hand-written passes with search.** Mirage jointly searches algebraic rewrites, schedules, and new custom kernels over a multi-level μGraph, finding fusions no fixed pass would (Wu *et al.*, 2024). For decode the prize is collapsing per-token launches, not a faster single matmul (Vellaisamy *et al.*, 2026).

- **The μGraph encodes residency by level.** Kernel graphs use device memory, block graphs use shared memory, thread graphs use registers, so the search optimizes bytes/token by construction (Wu *et al.*, 2024; Chen, 2026). The representation is the hardware model.
- **Verified megakernels attack launches/token directly.** MPK fuses an LLM forward pass into one launch via an SM-level task graph, reporting 1.2–6.7× latency reduction by removing per-operator host dispatch (Mirage Project contributors, 2025; NVIDIA, 2024). One launch per token is the limit case of this book's thesis.
- **Probabilistic equivalence makes discovered kernels deployable.** Checking candidates by polynomial identity testing over finite fields (LAX fragment) gives quantified correctness, so a fused kernel can replace a known-good path with confidence—but only within the verified fragment (Wu *et al.*, 2024). Know what the guarantee covers (Davies *et al.*, 2025).
- **Search cost and SKU-specificity are the trade-offs.** Superoptimization pays for hot, reused decode layers, not one-off graphs, and a μGraph tuned for one GPU is not optimal on another (Wu *et al.*, 2024; NVIDIA, 2022). Budget the search and re-tune per target (Kwon *et al.*, 2023).

Practical decision rules. Four rules summarize this chapter for a serving team. First, point superoptimization at hot, reused decode regions and keep conventional compilation for cold graphs, because search cost only amortizes over repeated execution (Wu *et al.*, 2024; Kwon *et al.*, 2023). Second, treat the verification report as a release artifact: record the fragment coverage, the confidence bound, and the reference it was checked against (Wu *et al.*, 2024; Davies *et al.*, 2025). Third, keep the reference path and telemetry in production behind a runtime switch so a discovered kernel can be rolled back without redeployment (Mirage Project contributors, 2025; Chen, 2026). Fourth, regenerate and re-verify artifacts whenever the model, GPU, driver, or served bucket changes, because nothing about a search result transfers without re-validation (Wu *et al.*, 2024; NVIDIA, 2022). Teams that apply these rules get the megakernel's latency win without giving up correctness; teams that skip them get a fast kernel they cannot debug (Vellaisamy *et al.*, 2026; Chen, 2026).

1.7 Conclusion

Mirage reframes compilation as multi-level superoptimization with verified correctness, and MPK shows its decode payoff: a single megakernel that collapses a token's launches and overlaps communication with computation inside one kernel, for 1.2–6.7× (Wu *et al.*, 2024; Mirage Project contributors, 2025). This is the intellectual source of the YiRage compiler used as the yardstick throughout the book—multi-level search, megakernel fusion, and residency encoded as checkable metadata—generalized beyond a single GPU artifact (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). The broader lesson is that the field is converging on search plus verification as the way to reach fewer, more resident launches, and that decode is the workload where this convergence pays off first because it has the least batch parallelism to fall back on (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Having now surveyed each compiler surface individually, Chapter **??** steps back to a unified analysis and a cross-hardware benchmark, comparing these stacks on the same decode metrics rather than on each project's favored microbenchmark (Chen, 2026; Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 3, 9, 13

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 13, 16

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Chen, T., Zheng, L., Yan, E., Jiang, Z., Moreau, T., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018a). Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*. AutoTVM; template-based statistical cost model (GBT/TreeGRU), transfer learning; 1.2–3.8x end-to-end. 3

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018b). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 3

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 3, 4, 6, 7, 10, 11, 12, 14, 15, 16

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 3, 13, 16, 18

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 6, 7, 11, 13, 14, 15, 16

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 3, 4, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16, 17

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 2, 8, 16, 18

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. `https://iree.dev`. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; `.vmfb` fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 2, 3

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 3, 8, 10, 11, 12, 17

Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 6

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. `https://github.com/mirage-project/mirage`. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; 1.2–6.7× latency reduction. 1, 2, 7, 8, 9, 11, 13, 14, 15, 17, 18

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 7, 9, 10, 14, 17

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 17

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 9, 16

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 3, 9

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μGraph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9×. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. (2020). Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879. Template-free auto-scheduler; hierarchical sampling, evolutionary search, task scheduler; up to 3.8x/2.6x/1.7x on Intel CPU/ARM CPU/NVIDIA GPU. 3

Index

Megakernel, 1
Mirage, 1

Superoptimization, 1